CSA1026- SOFTWARE ENGINEERING LAB EXPERIMENTS

EXP- 1

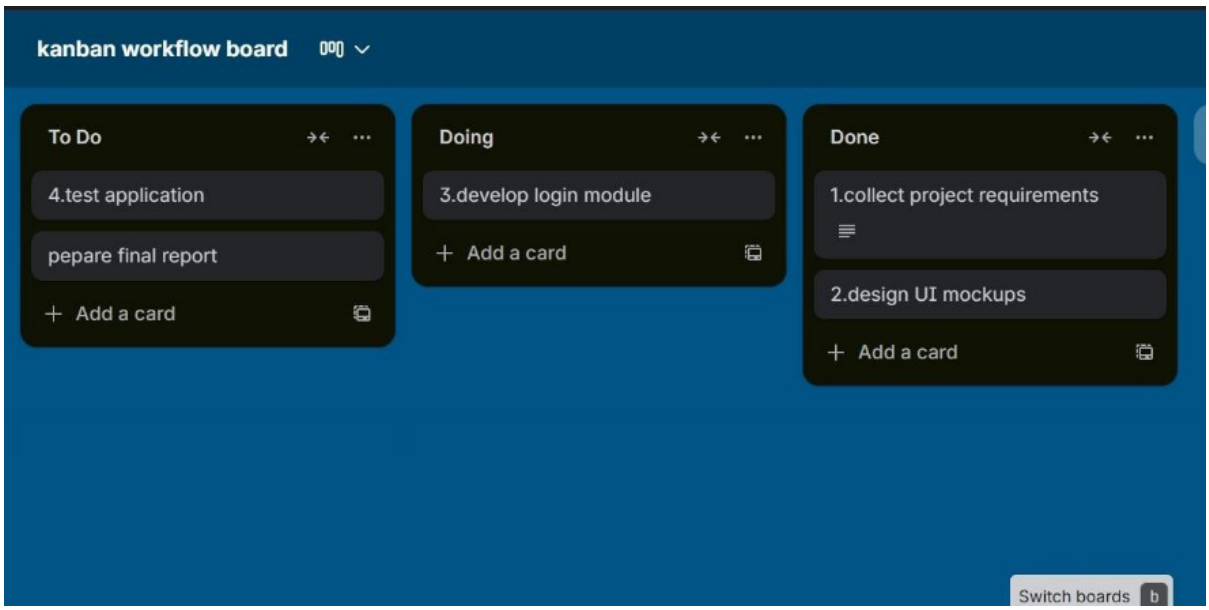
Create a Kanban Board (Jira/Trello)

Aim

To create and simulate a Kanban workflow using Jira or Trello.

Procedure

1. Login to Jira/Trello.
2. Create a new Kanban board.
3. Create columns: To Do, In Progress, Done.
4. Add at least 5 tasks.
5. Move tasks across columns to simulate workflow.
6. Observe task progress.



Result

Successfully created a Kanban board and simulated workflow by moving tasks across stages.

EXP- 2

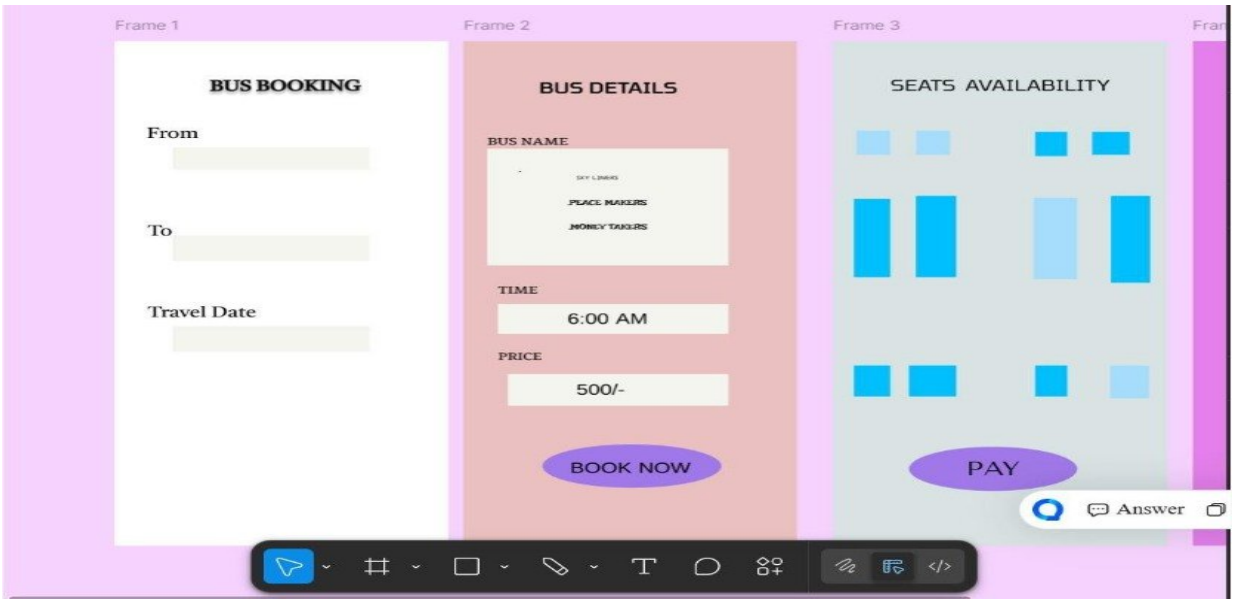
Bus Ticket Booking Prototype (Figma)

Aim

To design a simple Bus Ticket Booking System prototype using Figma.

Procedure

1. Open Figma and create a new design file.
2. Design screens:
    - Home Page
    - Search Bus
    - Seat Selection
    - Payment Page
3. Add navigation links.
4. Present prototype mode.



Result

A clickable prototype of Bus Ticket Booking System was created successfully.

EXP- 3

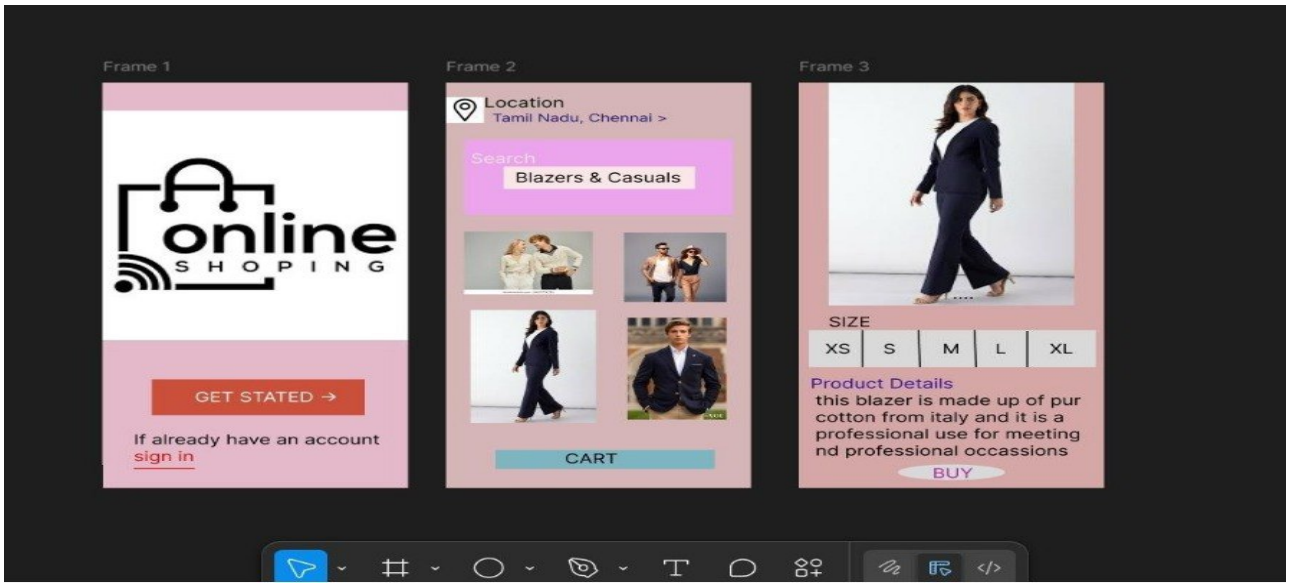
E- Commerce Mobile App Prototype (Figma)

Aim

To design a prototype for stakeholder feedback.

Procedure

1. Create mobile frame in Figma.
2. Design Home, Product Page, Cart, Checkout.
3. Create two UI variations (to resolve conflict).
4. Present to stakeholders for feedback.



Result

Prototype created and stakeholder feedback collected for UI finalization.

EXP- 4

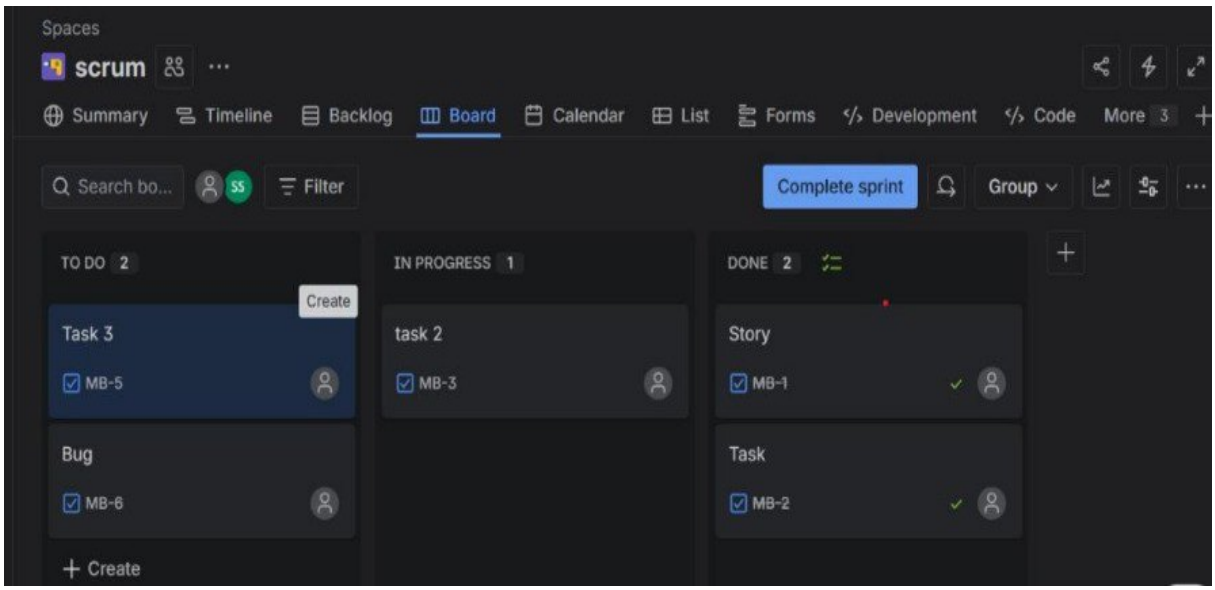
Scrum Project in Jira

Aim

To create and manage a Scrum project in Jira.

Procedure

1. Create Scrum project.
2. Add 5 backlog items.
3. Prioritize backlog.
4. Create 1- week sprint.
5. Move tasks to sprint.
6. Start sprint.
7. Update task status until completion.



Result

Scrum sprint executed successfully and sprint board tracked progress.

EXP- 5

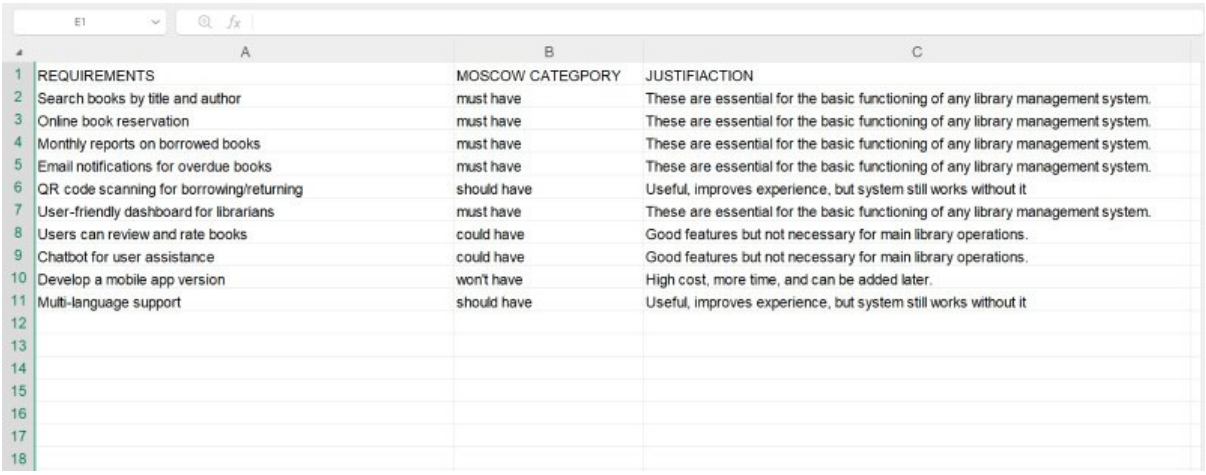
MoSCoW – Library Management System

Aim

To categorize requirements using MoSCoW method.

Procedure

1. List all requirements.
2. Analyze impact & feasibility.
3. Categorize into:
    - Must Have
    - Should Have
    - Could Have
    - Won't Have
4. Enter into Excel/Google Sheet.
5. Add justification column.



	A	B	C
1	REQUIREMENTS	MOSCOW CATEGPORY	JUSTIFIACTION
2	Search books by title and author	must have	These are essential for the basic functioning of any library management system.
3	Online book reservation	must have	These are essential for the basic functioning of any library management system.
4	Monthly reports on borrowed books	must have	These are essential for the basic functioning of any library management system.
5	Email notifications for overdue books	must have	These are essential for the basic functioning of any library management system.
6	QR code scanning for borrowing/returning	should have	Useful, improves experience, but system still works without it
7	User-friendly dashboard for librarians	must have	These are essential for the basic functioning of any library management system.
8	Users can review and rate books	could have	Good features but not necessary for main library operations.
9	Chatbot for user assistance	could have	Good features but not necessary for main library operations.
10	Develop a mobile app version	won't have	High cost, more time, and can be added later.
11	Multi-language support	should have	Useful, improves experience, but system still works without it
12			
13			
14			
15			
16			
17			
18			

Result

Requirements categorized successfully using MoSCoW prioritization.

EXP- 6

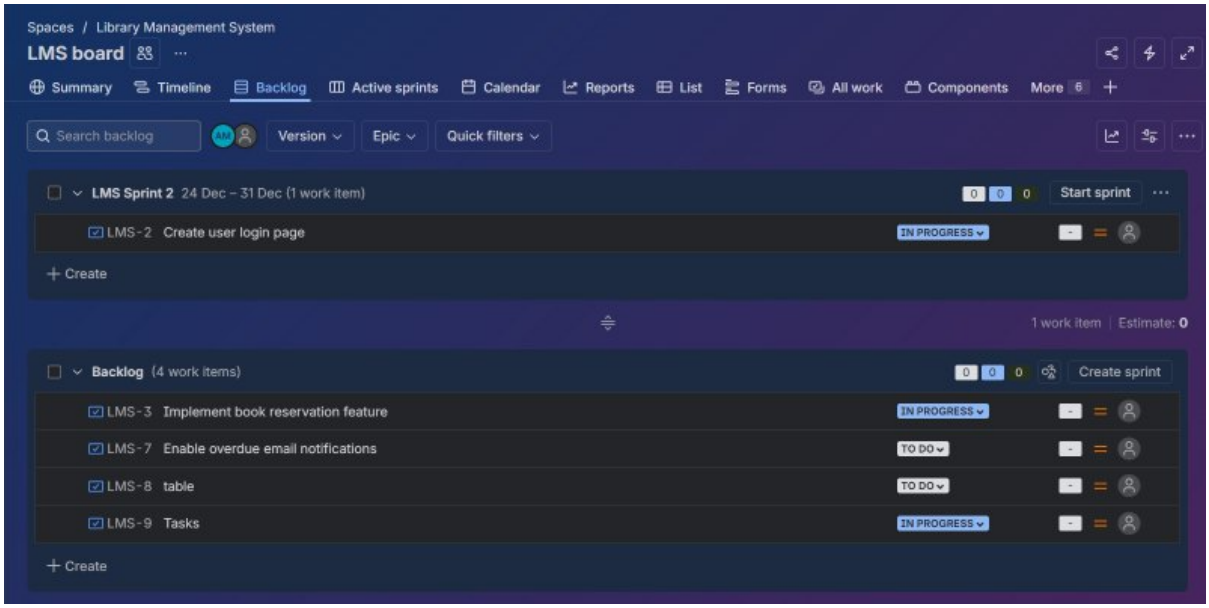
Link Jira with Confluence

Aim

To integrate Jira with Confluence for project tracking.

Procedure

1. Create Confluence page.
2. Insert Jira macro.
3. Embed 5 Jira issues.
4. Add progress bar.
5. Publish page.



Result

Jira issues embedded in Confluence with real- time progress tracking.

EXP- 7

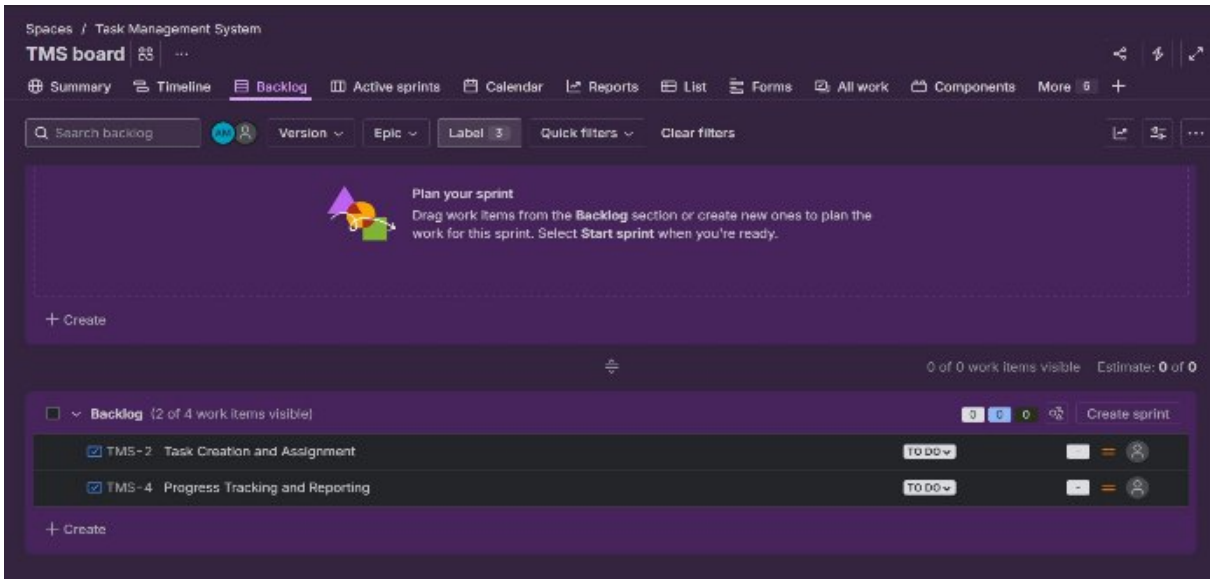
Task Management System – MoSCoW & Kano (Jira)

Aim

To prioritize system requirements using MoSCoW and Kano models.

Procedure

1. Create Jira project.
2. Add features as issues.
3. Categorize using MoSCoW.
4. Apply Kano model classification.
5. Document priority results.



Result

Requirements prioritized effectively using both models.

EXP- 8

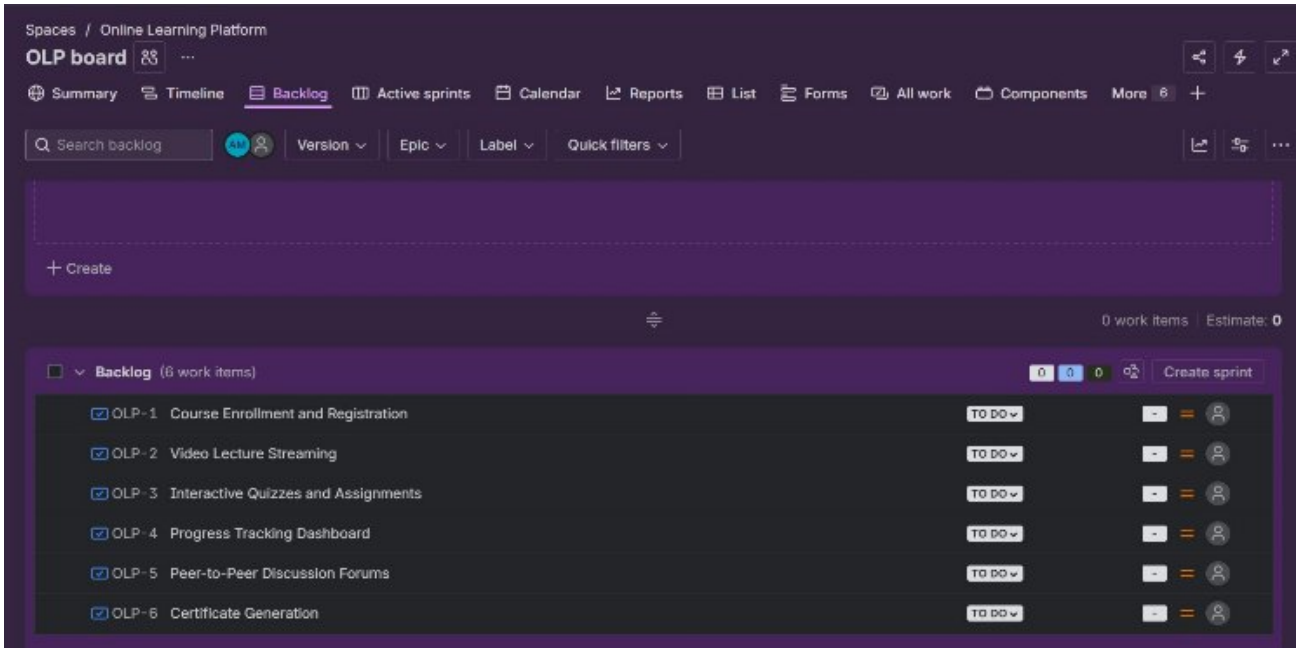
Online Learning Platform – MoSCoW & Kano

Aim

To categorize and prioritize platform requirements.

Procedure

1. Create Jira backlog.
2. Add all 6 features.
3. Categorize using MoSCoW.
4. Classify using Kano.
5. Prioritize accordingly.



Result

Features prioritized based on stakeholder value and satisfaction.

EXP- 9

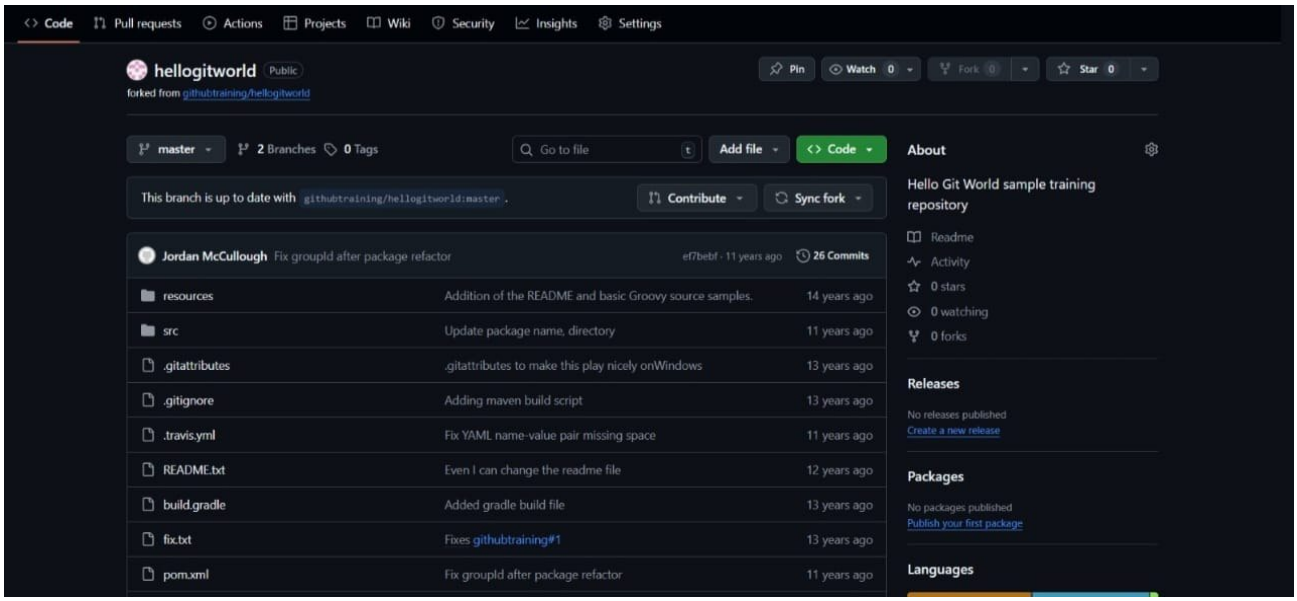
Fork & Pull Request Workflow (GitHub)

Aim

To demonstrate collaboration using fork and pull request workflow.

Procedure

1. Fork repository.
2. Clone locally.
3. Create feature branch.
4. Make changes.
5. Commit and push.
6. Create pull request.
7. Review & merge.



Result

Pull request created and successfully merged after review.

EXP- 10

Git Branching & Merge Conflict

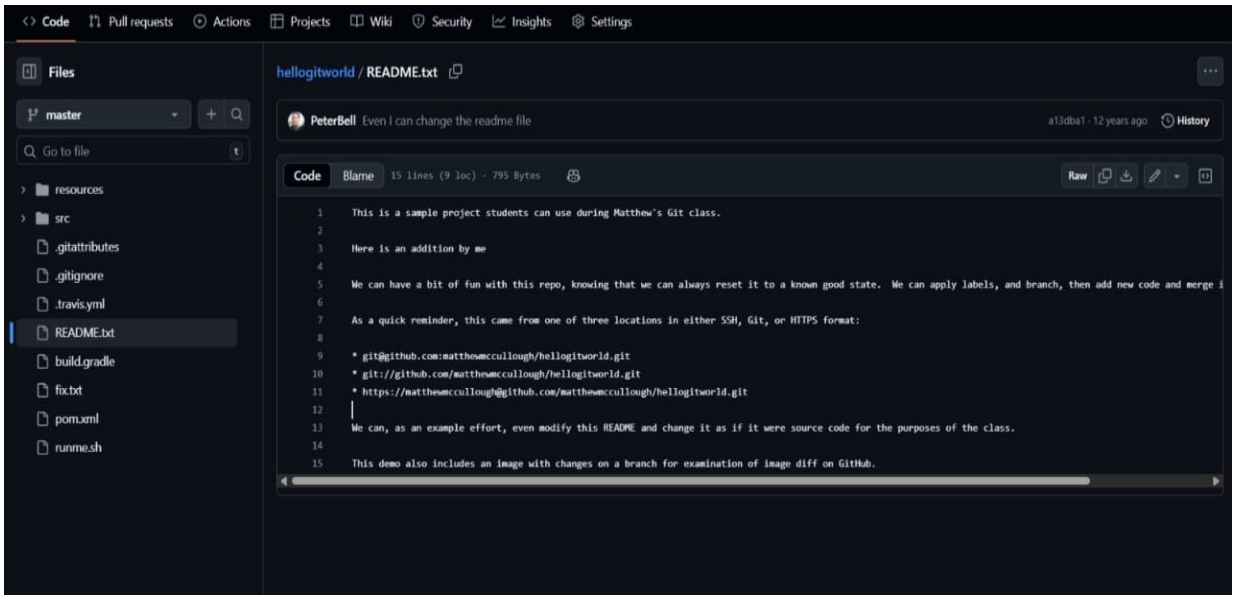
Aim

To resolve merge conflicts using Git.

Procedure

1. Clone repository.

2. Create feature branch.
3. Modify file.
4. Commit and push.
5. Pull latest main changes.
6. Merge main into feature branch.
7. Resolve conflicts.
8. Commit resolved changes.



Result

Merge conflicts resolved successfully and branch merged.

EXP- 11

Static Website with Docker

Aim

To containerize and serve a static website using Docker.

Procedure

1. Create index.html.
2. Create Dockerfile with Nginx.

3. Build Docker image.
4. Run container.
5. Access via browser.

Code Blame Raw

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>My Static Website</title>
</head>
<body>
    <h1>Welcome to My Static Website!</h1>
    <p>This static site is served using Docker and Nginx.</p>
</body>
</html>
```

Result

Static website successfully deployed in Docker container.

EXP- 12

Flask API with Docker & Kubernetes

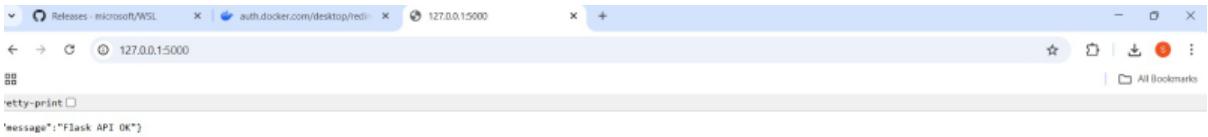
Aim

To deploy a Flask API using Docker and Kubernetes.

Procedure

1. Create Flask app.
2. Write Dockerfile.
3. Build & push image.

4. Create Deployment & Service YAML.
5. Deploy using kubectl.
6. Access via NodePort.



Result

Flask API successfully deployed on Kubernetes cluster.

EXP- 13

CI/CD using Jenkins

Aim

To automate build and deployment using Jenkins.

Procedure

1. Install Jenkins.
2. Create Dockerfile.
3. Write Jenkinsfile.
4. Configure webhook trigger.

5. Run pipeline.

Result

CI/CD pipeline executed automatically on code commit.

EXP- 14

Continuous Deployment with GitHub Actions

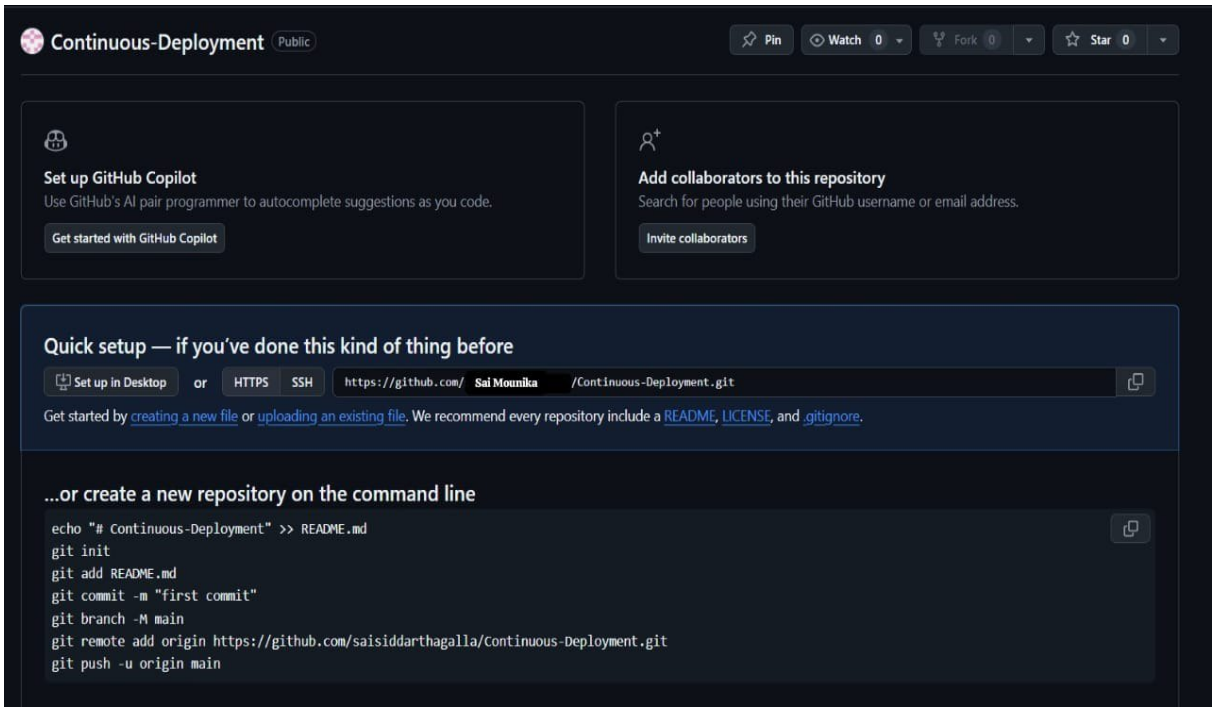
Aim

To automate Docker build and deployment using GitHub Actions.

Procedure

1. Create GitHub repo.
2. Add Dockerfile.
3. Create workflow YAML.
4. Push code.

5. Verify auto deployment.



Result

Application deployed automatically via GitHub Actions pipeline.

EXP- 15

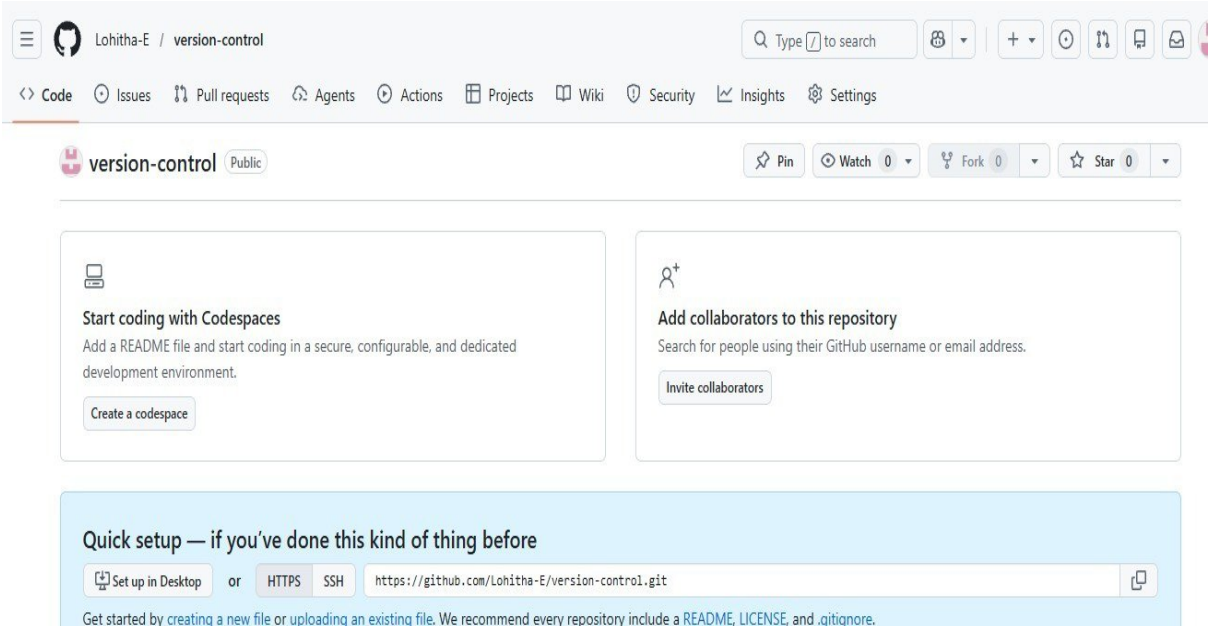
GitHub Version Control

Aim

To implement version control workflow.

Procedure

1. Create repository.
2. Create branches.
3. Make changes.
4. Create pull requests.
5. Resolve conflicts.
6. Document in README.



Result

Version control workflow implemented successfully.

EXP- 16

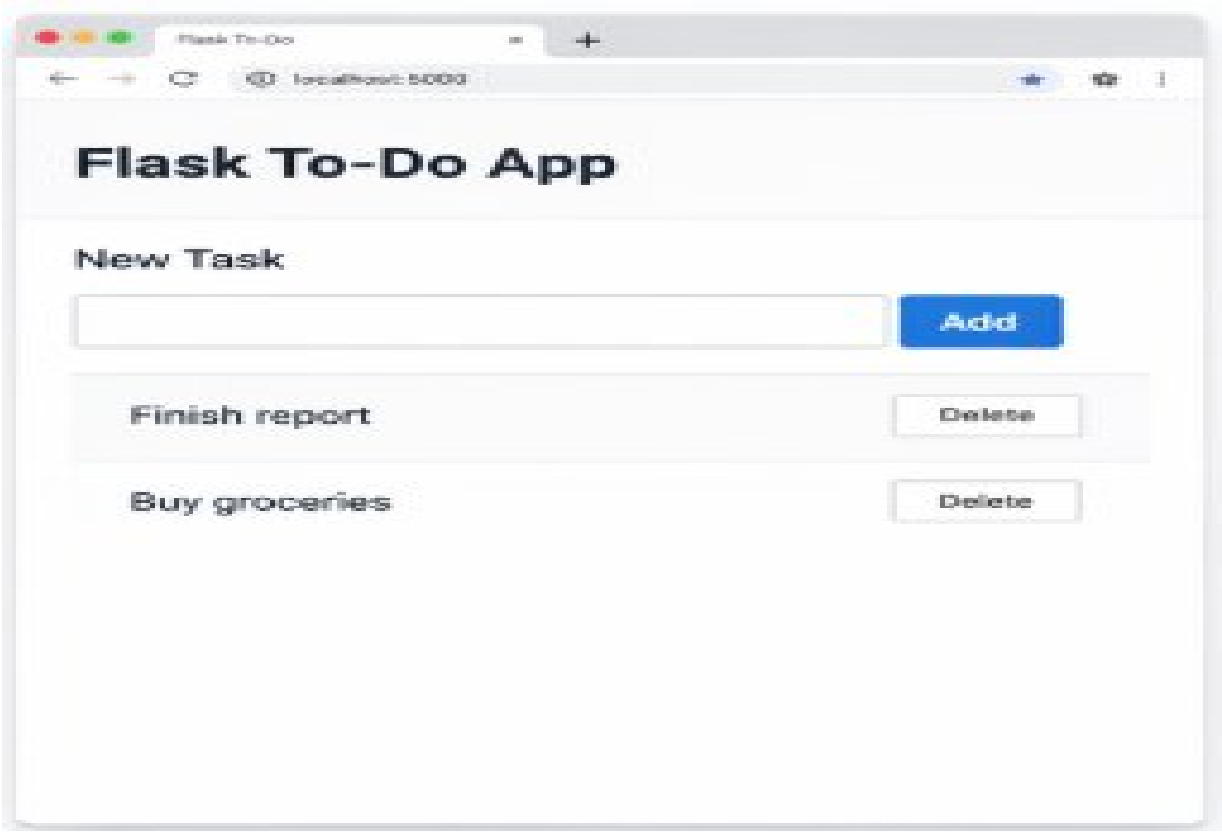
Containerize Flask To- Do App

Aim

To containerize Flask To- Do application.

Procedure

1. Write Flask app.
2. Create Dockerfile.
3. Build image.
4. Run container.
5. Push to Docker Hub.



Result

Flask To- Do app successfully containerized and deployed.

EXP- 17

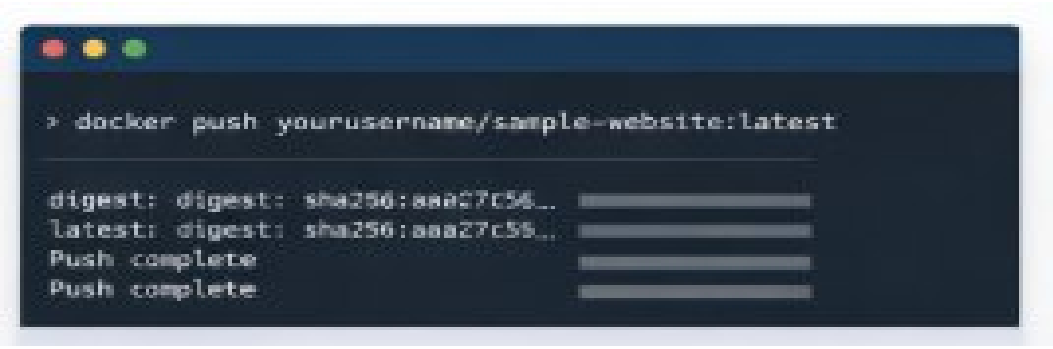
Push & Pull Docker Image

Aim

To demonstrate Docker image push and pull.

Procedure

1. Create Docker image.
2. Tag image.
3. Push to Docker Hub.
4. Pull on another system.
5. Run container.



Result

Docker image successfully pushed and pulled.

EXP- 18

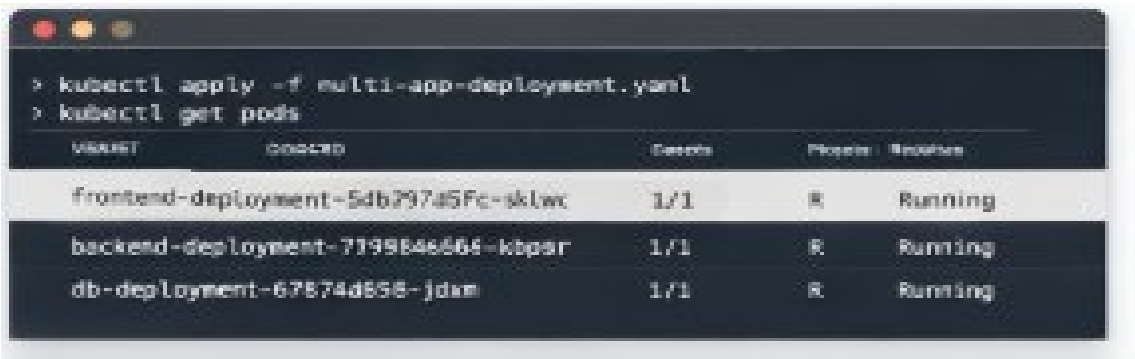
Multi- Container App with Kubernetes

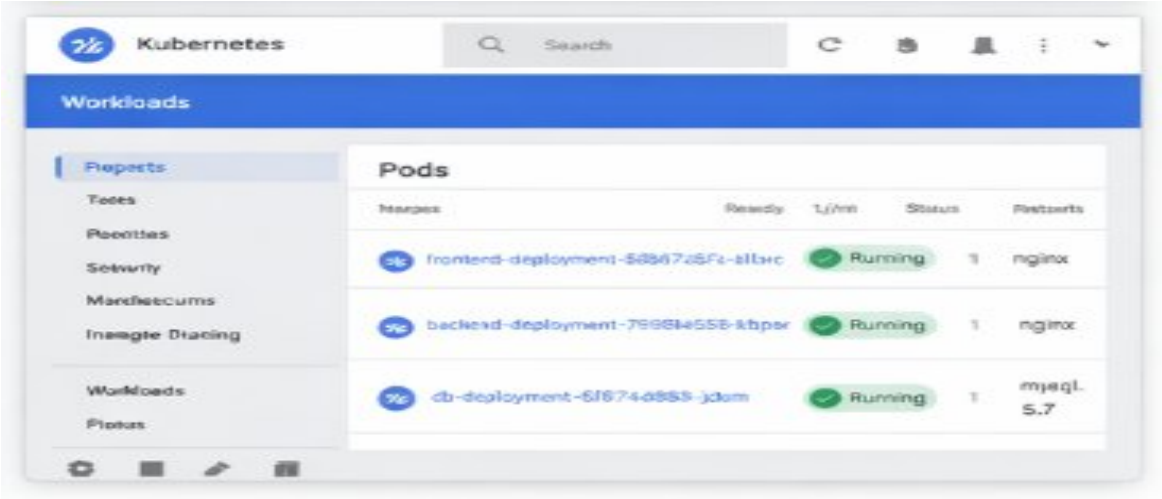
Aim

To deploy multi- container app using Kubernetes.

Procedure

1. Create frontend, backend, database containers.
2. Write YAML files.
3. Deploy using kubectl.
4. Monitor pods.
5. Scale frontend.





Result

Multi- container application deployed and scaled successfully.

EXP- 19

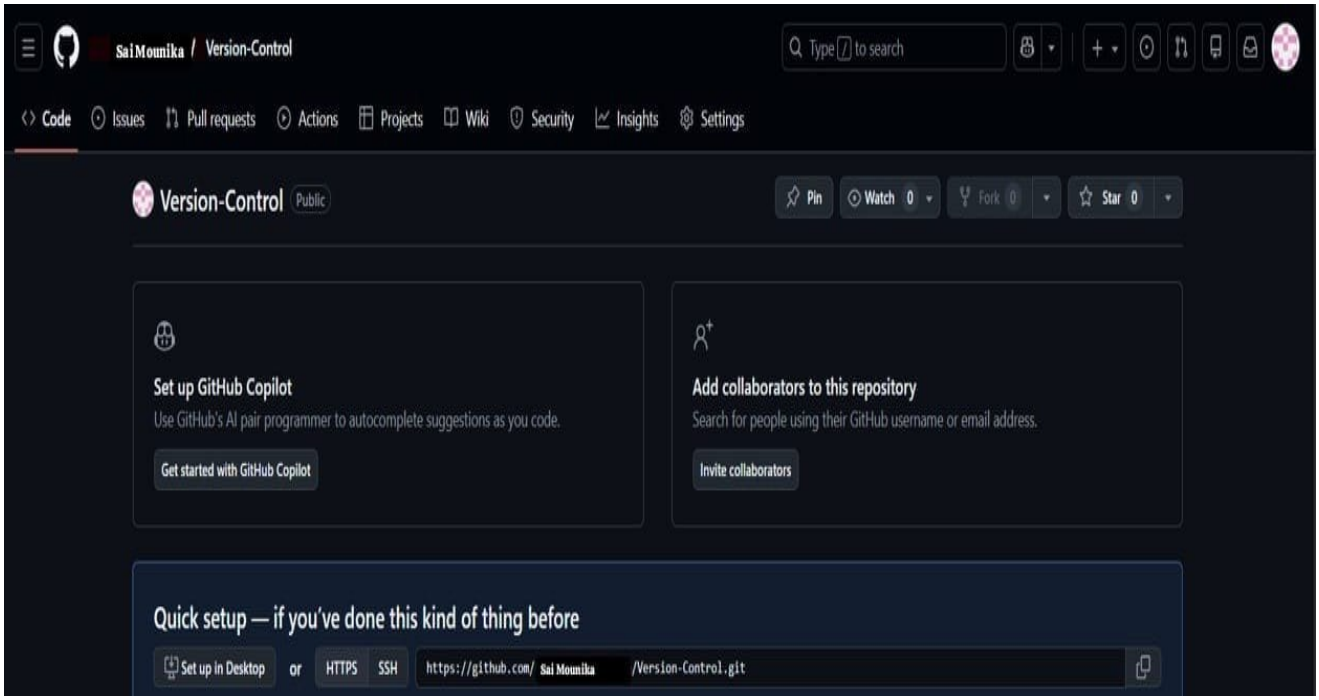
CI/CD with GitHub Actions

Aim

To automate testing and deployment using GitHub Actions.

Procedure

1. Create workflow YAML.
2. Add build & test steps.
3. Push code.
4. Verify deployment.



Result

CI/CD pipeline executed successfully.

EXP- 20

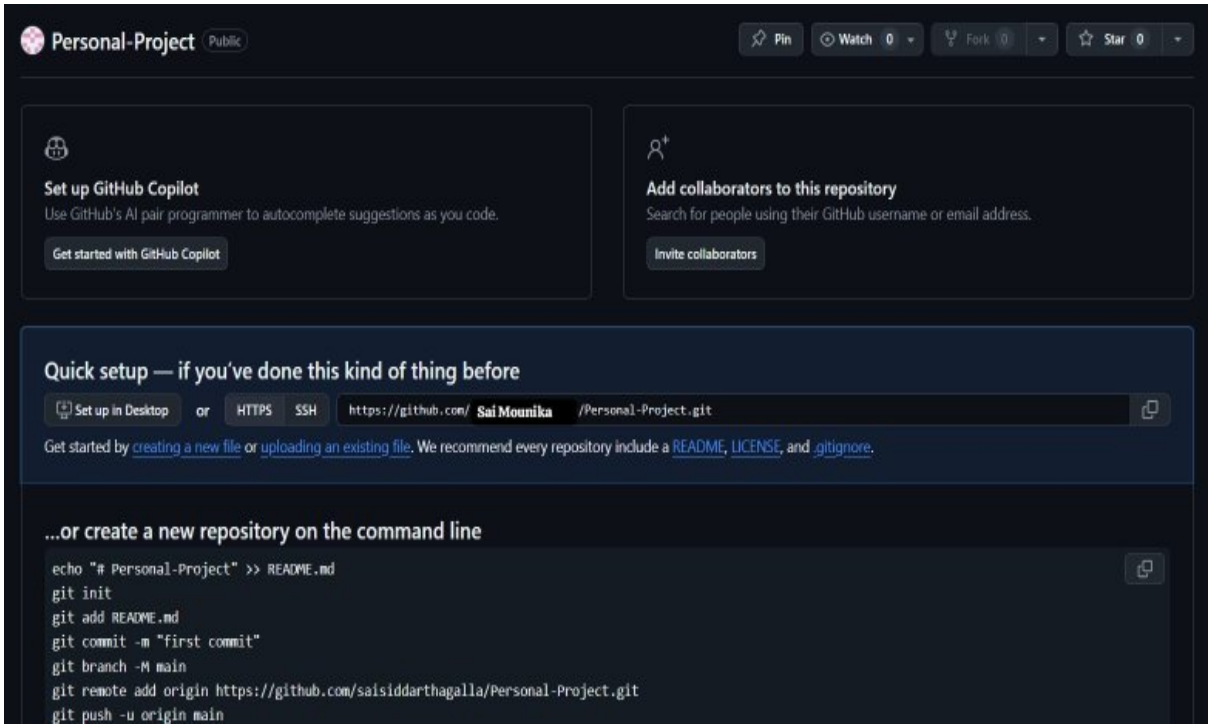
Create GitHub Repository

Aim

To create and update a GitHub repository.

Procedure

1. Create repository.
2. Clone locally.
3. Edit README.
4. Commit & push.



Result

Repository updated successfully.

EXP- 21

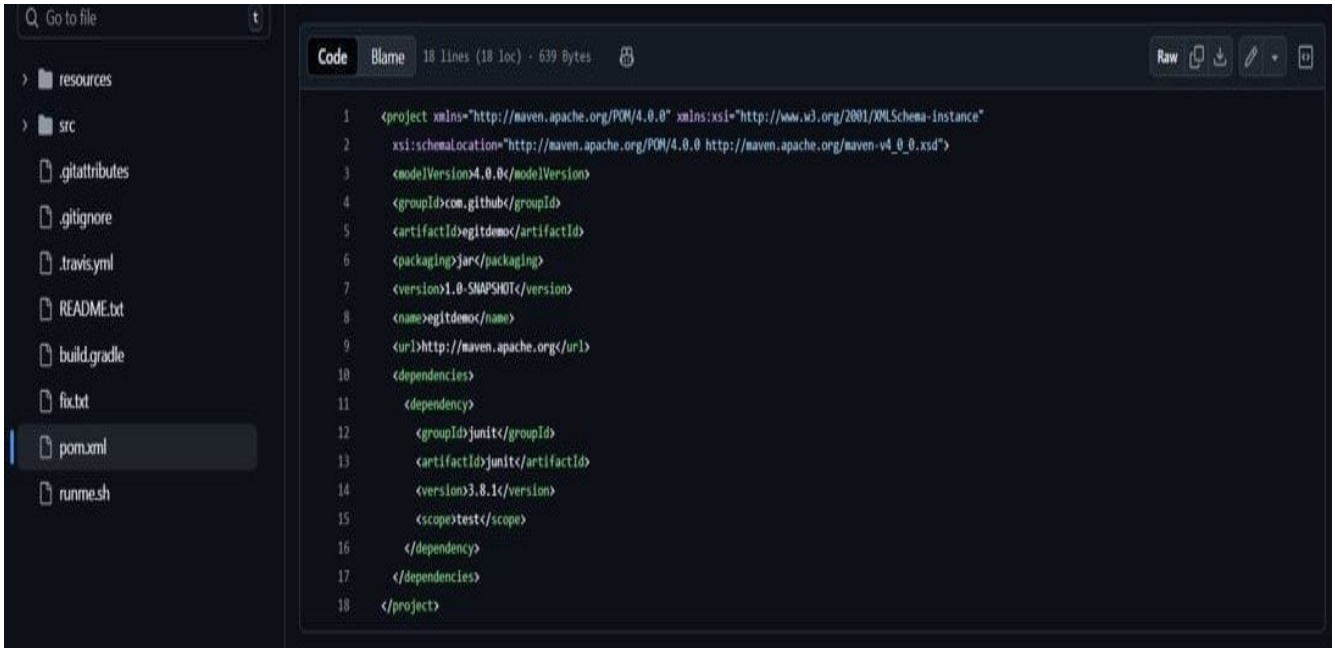
Clone & Modify Repository

Aim

To clone and edit a GitHub repository.

Procedure

1. Clone repo.
2. Open file.
3. Edit content.
4. Save changes.



Result

File successfully modified locally.

EXP- 22

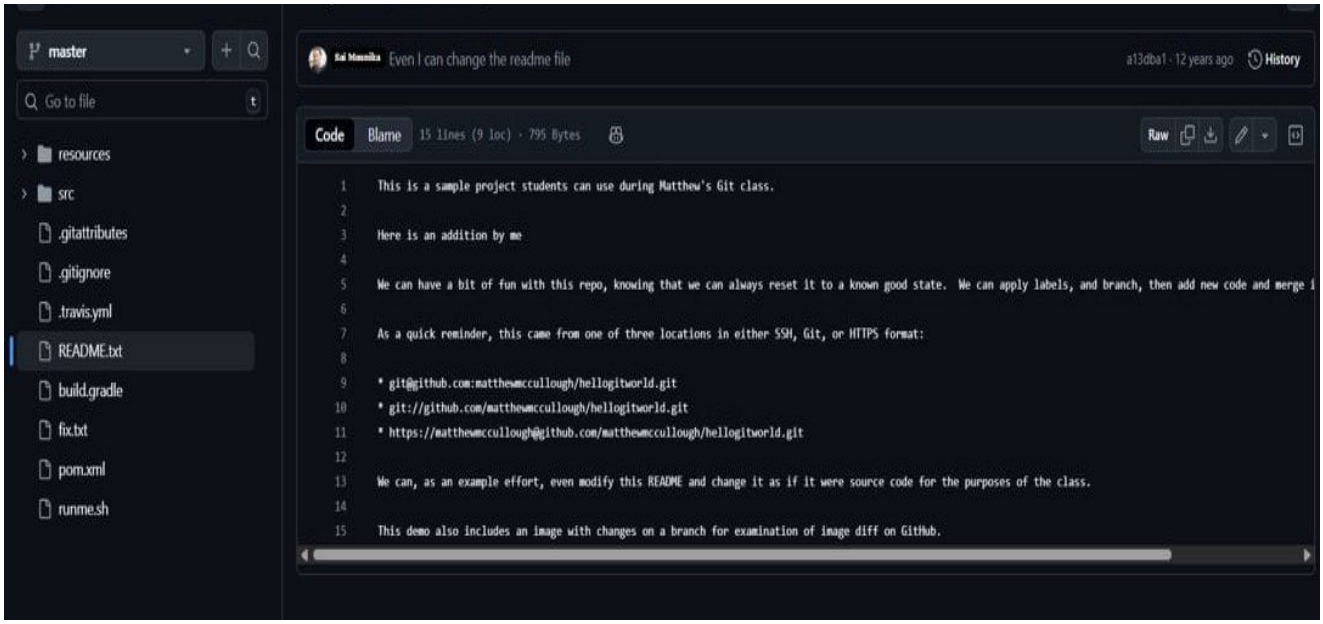
Commit & Push Changes

Aim

To commit and push changes to GitHub.

Procedure

1. Stage changes.
2. Commit with message.
3. Push to GitHub.



Result

Changes successfully reflected in GitHub repository.

EXP- 23

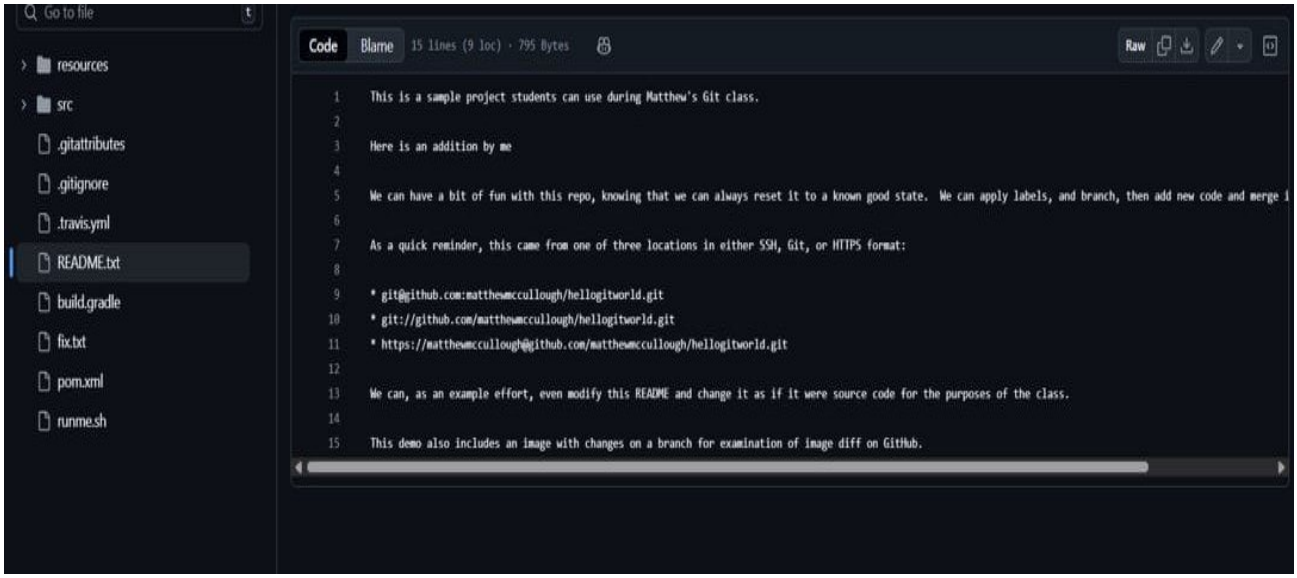
Pull Latest Changes

Aim

To update local repository with remote changes.

Procedure

1. Clone repo.
2. Collaborator pushes changes.
3. Run git pull origin main.
4. Verify updates.



Result

Local repository updated successfully.

EXP- 24

Create feature- login Branch

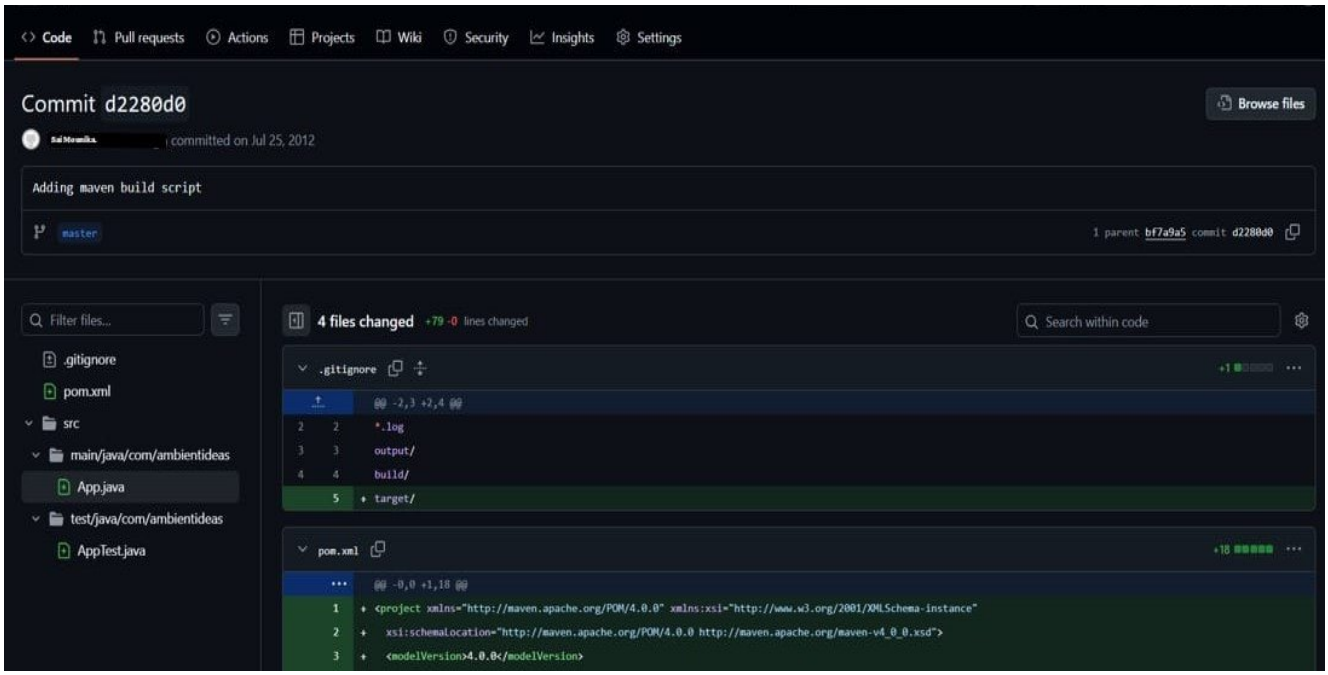
Aim

To implement login feature using Git branching.

Procedure

1. Create branch feature- login.
2. Add login.py.
3. Commit & push.
4. Create pull request.

5. Merge branch.



Result

Login feature successfully merged into main branch.

EXP- 25

Fork & Implement Feature

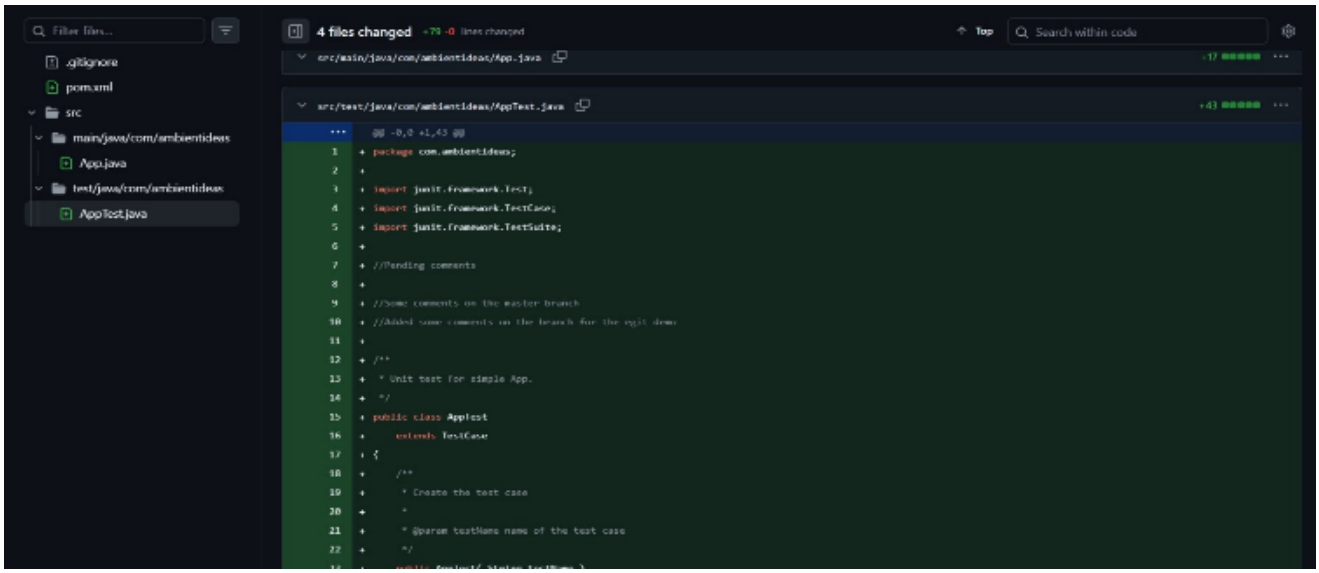
Aim

To collaborate using fork- based workflow.

Procedure

1. Fork repository.
2. Clone locally.
3. Create new branch.
4. Add feature.
5. Push to fork.

6. Create pull request.



Result

Feature successfully contributed via pull request.